Clasificación mediante K Vecinos más Cercanos (K-Nearest Neighbors) y embeddings de texto – Dataset E-Commerce / Call center

Presentado por

Cristian Camilo Posada García

Laura Camila Gutiérrez Hernández

Presentado a

Ing. Alexander Espinosa García

Universidad de Cundinamarca, Extensión Facatativá

Ingeniería de Sistemas y Computación

Profundización I – Machine Learning

8 de Abril de 2026

Tabla de contenido

Descripción del dataset 3

Generación de embeddings 3

Métricas de similitud 6

- **Similitud de coseno** 6
- **Distancia Manhattan (L1)** 8
- **Distancia Euclidiana (L2)** 9
- **Distancia de Minkowski** 11

Algoritmo KNN 12

Evaluación Hold-Out 16

Gráficas 19

Ejecución del algoritmo y pruebas 23

Conclusiones 29

Bibliografía 30

Link al colab: https://colab.research.google.com/drive/1NU3PYZq_G1_8saejbTKZ__fC-WIc-kgl?usp=sharing#scrollTo=jtB5WyiAQn3Q

Link repositorio Laura: https://github.com/Lauri06/Algoritmo-KNN---Machine-Learning

Link repositorio Cristian: https://github.com/CRICAPOGA/Algoritmo-KNN---Machine-Learning.git

Descripción del dataset

El dataset cuenta con un total de 600 registros, con dos columnas, la primera se denomina **texto** y se refiere a la inconformidad que manifiesta el cliente, respecto a su compra en línea, la segunda columna denominada **clase**, se refiere a los tipos de reclamación y existen 6 clases:

1. *consulta_producto*
2. *devolucion_reembolso*
3. *problema_pago*
4. *producto_danado*
5. *reclamo_entrega*
6. *seguimiento_pedido*

El código lee el archivo csv y separa las dos columnas, por un lado, deja los textos de las reclamaciones y los almacena en una lista denominada textos, separando cada una de las frases con comas y entre comillas simples. Estas frases en su mayoría hablan sobre pedidos que no han sido entregados, compras que no se pudieron realizar porque no acepto el método de pago, personas que desean saber el estado del envío y productos que no cumplen con las expectativas de los clientes o llegaron dañados, indicando a su vez la ciudad donde se realiza el envío.

Generación de embeddings

Según Barnard J. de IBM los embeddings son una herramienta fundamental para el machine learning ya que estas incrustaciones permiten a los modelos encontrar objetos similares, esto funciona porque se toman ya sean textos como en el caso de este dataset que

son peticiones de clientes, o también puede ser imágenes o audio. Se toman estas entradas como puntos en un espacio vectorial continuo y cada una de las ubicaciones de estos puntos cuenta con un significado semántico. En el caso de los textos los embeddings permiten al modelo entender de manera fácil las relaciones que existen entre las palabras y su categoría, de esta forma el modelo no tiene que analizar una a una las palabras de una manera aislada.

Las entradas se representan en vectores o matrices numéricas, donde cada número indica donde se encuentra un objeto en cierta dimensión, se puede llegar a tener más de mil dimensiones, todo esto depende de los datos de entrada. Entonces, la cercanía entre las incrustaciones en ese espacio de n-dimensiones indica que tan similares son una de la otra, para calcular la longitud de los espacios vectoriales se usan técnicas como coseno o Minkowski que se explicarán más adelante.

Figura 1

Generación de Embeddings en el código

```
def generar_embeddings(modelo, textos):
  """Convierte una lista de textos en vectores numéricos (embeddings)"""
  embeddings = modelo.encode(textos, show_progress_bar=True)
  return [list(map(float, v)) for v in embeddings]
```

```
modelo = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
embeddings = generar_embeddings(modelo, textos)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100% 229/229 [00:00<00:00, 16.3kB/s]
config_sentence_transformers.json: 100% 122/122 [00:00<00:00, 11.7kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fas
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
README.md: 3.89k/? [00:00<00:00, 286kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.51kB/s]
config.json: 100% 645/645 [00:00<00:00, 49.7kB/s]
model.safetensors: 100% 471M/471M [00:03<00:00, 382MB/s]
Loading weights: 100% 199/199 [00:00<00:00, 484.15it/s, Materializing param=pooler.dense.weight]
```

Mediante la función *generar_embeddings* que recibe como parámetros el modelo para la creación de embeddings y la lista con los textos de las reclamaciones de los clientes, convierte esa lista en vectores numéricos, mediante el método .encode() que toma los textos y los pasa por la red neuronal del transformer, la cual analiza la semántica y el contexto de cada una de las frases.

Con la línea de código *[list(map(float, v)) for v in embeddings]* se asegura que cada número que se encuentra en el vector sea un número decimal o un float, porque este tipo de modelos devuelven datos de tipo float32 que son muy útiles para realizar cálculos matemáticos rápidos, pero son difíciles de guardar en bases de datos, por esta razón es necesario asegurar que los números sean de tipo float y finalmente con list se convierte el vector en una lista, por lo que se obtiene una lista de listas, es decir cada lista dentro de la lista es el embedding de cada una de las reclamaciones.

Se utiliza el modelo *paraphrase-multilingual-MiniLM-L12-v2* el cual funciona como un traductor de texto a matemáticas. El modelo como primer paso rompe cada una de las frases y las almacena como tokens y al ser multilingüe entiende independientemente del idioma que ciertas palabras se refieren a un mismo concepto, es decir el modelo entiende que pedido y order, significan lo mismo y no las trata como palabras distintas. El modelo también analiza el contexto de cada palabra, no analiza solo la palabra pedido, sino que también analiza las palabras que se encuentran alrededor.

Al generar los embeddings el modelo comprime toda la información de cada una de las frases en un vector o lista de números decimales y a partir de esto por la distancia entre cada uno de los vectores se puede identificar si dos frases tienen un significado similar.

Métricas de similitud

Para evaluar si dos frases tienen un significado similar es necesario medir la distancia entre sus vectores, por lo que es necesario aplicar alguna métrica, a continuación, se explicaran cada una de las métricas implementadas en el código.

Similitud de coseno

Según IBM esta métrica de similitud muestra la semejanza entre dos puntos teniendo en cuenta la dirección hacia la que apuntan esta métrica no tiene en cuenta la longitud o el tamaño de los vectores. La implementación de la similitud de coseno resulta eficaz para espacios de grandes dimensiones donde las métricas basadas en la distancia o mejor conocidas como tradicionales llegan a tener dificultades.

Para hacer uso de esta métrica es necesario calcular el coseno del ángulo de dos vectores que son diferentes de 0. Este cálculo genera resultados entre un intervalo de -1 y 1, para el primer caso donde el resultado es -1 indica que los dos vectores apuntan a direcciones totalmente opuestas, en el segundo caso donde la similitud es de 0 indica que los vectores son ortogonales, es decir que los vectores forman un ángulo recto (90°) y a su vez se entiende que son perpendiculares, por lo que los vectores no tendrían similitud en su dirección. Finalmente, en el caso 3, cuando el resultado es 1 indica que los vectores apuntan hacia la misma dirección, es decir que tienen similitud en su dirección.

Para realizar una comparación semántica se usan los embeddings generados con la función anterior y la similitud de coseno para evaluar la cercanía semántica que hay entre las entradas.

Figura 2

Función similitud de coseno

Similitud Coseno

$$\cos(\theta) = \frac{v_1 \cdot v_2}{||v_1|| \cdot ||v_2||}$$

```python
def similitud_coseno(v1, v2):
    """
    Similitud del coseno entre dos vectores
    Devuelve un valor en [-1, 1]; 1 = identicos
    """
    producto = sum(a * b for a, b in zip(v1, v2))
    norma1 = sum(a ** 2 for a in v1) ** 0.5
    norma2 = sum(b ** 2 for b in v2) ** 0.5
    if norma1 == 0 or norma2 == 0:
        return 1.0 # Maxima distancia si alguno es nulo
    similitud = producto / (norma1 * norma2)
    return 1.0 - similitud # Convertir a distancia (menor = más cercano)
```

Esta función tiene dos parámetros de entrada v1 y v2, que son los vectores o matrices numéricas de las dos frases que se van a comparar, primero se calcula el producto punto entre los vectores, con el uso de la función zip se toma el primer elemento de cada vector o lista y los agrupa, luego de esta agrupación cada una de las parejas o agrupación del zip se multiplica para finalmente sumar cada uno de los resultados, obteniendo así el producto punto. Con norma1 y norma2 se está calculando la magnitud o longitud de cada vector haciendo uso de del teorema de Pitágoras, que es la raíz de la suma de cada uno de los cuadrados. Después se realiza una validación para los casos en que la magnitud sea 0, porque no se puede realizar una división por cero, entonces se le asigna el valor de 1.0 y de esta forma se entiende que en caso de que sea un texto vacío o el vector solo tenga 0, se le asigna una máxima distancia.

Distancia Manhattan (L1)

Según Fletcher, 2003 la distancia Manhattan es útil porque permite calcular distancias en forma de cuadriculas o entre puntos en un espacio de n dimensiones. Esta métrica se diferencia de la distancia euclidiana porque esta mide la línea más corta entre dos puntos, en cambio la distancia Manhattan mide la suma de las diferencias entre las coordenadas de los puntos. En esta métrica no existen las diagonales ya que solo se puede mover hacia arriba y hacia abajo y de derecha a izquierda, simulando así la forma en que se recorrerían las calles de una ciudad cuadriculada como Nueva York.

La distancia Manhattan en un espacio de n dimensiones es la suma de las diferencias absolutas de las coordenadas cartesianas, es importante recalcar que el uso del valor absoluto permite garantizar que todas las mediciones de distancia sean positivas, porque no puede existir una distancia negativa, independientemente de la dirección del desplazamiento esta métrica refleja la verdadera distancia escalar.

Figura 3

Función para el cálculo de la distancia Manhattan

Distancia Manhattan

$$L_1 = \sum |v1_i - v2_i|$$

```
def distancia_l1(v1, v2):
    """
    Rango: [0, infinito) Cuanto menor, mas similares
    """
    return sum(abs(a - b) for a, b in zip(v1, v2))
```

Al igual que en la similitud de coseno se hace uso de la función zip para emparejar las coordenadas de los puntos es decir $a1\ con\ b1$ hasta n. A diferencia de la función anterior a la diferencia entre a -b se le calcula el valor absoluto para no obtener distancias negativas, porque en esta métrica lo que importa es cuantas unidades de distancia hay entre a y b. Finalmente se suman cada una de las distancias calculadas, en base al resultado se puede inferir que si este es cercano a 0 por ejemplo entre dos palabras como paquete y pedido se infiere que son similares y muy probablemente son sinónimos. Si el número es muy alto quiere decir que las frases o palabras no tienen relación o su contexto es diferente.

Distancia Euclidiana (L2)

Esta métrica representa al camino más corto entre dos puntos en el espacio euclidiano, este puede ser en dos dimensiones o más. Se basa en el teorema de Pitágoras donde la hipotenusa de un triángulo rectángulo es igual a la suma de ambos lados elevados al cuadrado. Cuando se desea calcular la distancia en un espacio bidimensional la fórmula se define la siguiente manera:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Donde *d* es la distancia euclidiana entre dos puntos, es decir el segmento de línea que conecta a esos dos puntos. x_1, y_1 se refiere a la coordenada del primer punto y x_2, y_2 es la coordenada del segundo punto. Si se desea realizar el cálculo en tres dimensiones solo basta con agregar $z_2\ y\ z_1$ respectivamente. En esta actividad se cuenta con más de tres dimensiones por lo que es necesario hacer uso de la siguiente fórmula, que es la misma que se aplicará en el código.

$$d = \sqrt{\sum_{i=1}^{n} (x_{2i} - x_{1i})^2}$$

Por medio de esta fórmula se puede calcular la distancia euclidiana entre dos puntos en un espacio de n dimensiones. El entendimiento es similar al cálculo en dos dimensiones. En este caso i varía desde 1 hasta n, permitiendo así obtener las coordenadas de cada punto en la cantidad de dimensiones acordada (n). $x_{11}, x_{12}, x_{13}, \ldots\ x_{1n}$ representa la coordenada del primer punto y $x_{21}, x_{22}, x_{23}, \ldots\ x_{2n}$ es la coordenada en el punto 2.

Figura 4

Función cálculo distancia Euclidiana

Distancia Euclidiana

$$L_2 = \sqrt{\sum (v1_i - v2_i)^2}$$

```python
def distancia_l2(v1, v2):
    """
    Rango: [0, infinito) Cuanto menor, mas similares
    """
    return sum((a - b) ** 2 for a, b in zip(v1, v2)) ** 0.5
```

Como se puede observar en la figura 4 se está haciendo uso de la misma función que se explicó anteriormente, en el código se utiliza la función zip para emparejar los elementos de cada vector, que en terminología de la explicación anterior serían las coordenadas de un punto, después se calcula la diferencia entre cada coordenada y se eleva al cuadrado, como se encuentra en un bucle for, cada una de las diferencias se van sumando para finalmente sacar la raíz cuadrada, como no se puede usar math, solo se toma el

resultado y eleva a 0.5 que es ½, lo cual representa a la raíz cuadrada y así se obtiene el resultado. De esta forma se puede identificar la cercanía que hay entre los datos, si el resultado es más cercano a cero quiere decir que son similares.

Distancia de Minkowski

Es una forma de medir la trayectoria ya sea recta o curva entre dos puntos. Esta métrica es la generalización de varias medidas de distancia, esta permite medir la distancia entre dos puntos en un espacio de n dimensiones, utiliza un parámetro p que permite la adaptación a diferentes espacios o características de los datos. Permite alternar entre la distancia Manhattan y la distancia Euclidiana según la necesidad, además reconoce que hay conjuntos de datos que no se adaptan bien a una sola métrica, que realmente se requiere de una herramienta que permita alternar entre métricas como los es la distancia de Minkowski. La fórmula general es la siguiente:

$$d = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{\frac{1}{p}}$$

Donde x y y son dos puntos en un espacio de n dimensiones, p es un parámetro que permite determinar el tipo de distancia, además este siempre debe ser mayor o igual a 1. Este parámetro se elige por prueba y error, se prueban diferentes valores de p, con el fin de determinar qué valor proporciona el mejor rendimiento según cada conjunto de datos. Si se escoge p=1 la distancia de Minkowski se convierte en la distancia de Manhattan y si se usa p=2, esta se convierte en la distancia euclidiana. A continuación, teniendo en cuenta la figura 5 se explicará la implementación de esta métrica en código:

Figura 5

Función cálculo distancia de Minkowski

Distancia Minkowski

$$L_p = \left(\sum |v1_i - v2_i|\right)^{\frac{1}{p}}$$

```
def distancia_minkowski(v1, v2, p=3):
  """
  Generalizacion: p=1 -> Manhattan, p=2 -> Euclidiana
  Se usa p=3 como norma elegida
  """
  return sum(abs(a - b) ** p for a, b in zip(v1, v2)) ** (1 / p)
```

La implementación de esta función es muy parecida a las anteriores, se emparejan usando zip, luego se calcula la diferencia absoluta entre cada dimensión, pero en este caso se eleva a la potencia p, en esta actividad haremos uso de p=3, ya que tenemos las funciones para el cálculo de distancia de Manhattan y de Euclides, entonces no tiene sentido usar p=1 o p=2. Luego se suman cada uno de los valores luego de aplicar la potencia de p y a esa suma se le saca la raíz que en este caso sería la raíz cúbica. En este caso al usar p=3 se busca tener una coincidencia casi exacta en cada una de las dimensiones, en caso de que exista una diferencia notable se considerará como alejado.

Algoritmo KNN

Según Kavlakoglu E. de IBM el algoritmo KNN es un clasificador de aprendizaje supervisado, que utiliza la proximidad para realizar las clasificaciones. Este algoritmo puede resolver varios tipos de problemas como regresión, pero su uso más común es el de clasificación, donde se encuentran puntos similares que se encuentran a una distancia

pequeña uno del otro. Este algoritmo identifica los vecinos más cercanos a un punto determinado y se le asigna una etiqueta de clase, para determinar la distancia entre el punto de consulta y los demás puntos (vecinos) se utilizan métricas o medidas de distancia, como la distancia euclidiana, la distancia Manhattan o la distancia de Minkowski que ya se explicaron y se implementaron en el código previamente.

Antes de ejecutar el algoritmo es necesario definir el valor de k, este valor define cuantos vecinos se verificarán para clasificar adecuadamente un punto de consulta especifico. La definición de k no tiene una regla estricta, se recomienda que sean números impares con el fin de evitar empates en la clasificación, pero realmente la elección es un acto de equilibrio, porque depende al valor escogido este puede conducir a un sobreajuste o un ajuste que no es suficiente, es decir, con k siendo un numero pequeño generalmente se obtiene una varianza alta pero un sesgo bajo, en cambio cuando k es un número grande el sesgo es mayor pero su varianza es mas pequeña, entonces, la elección del valor de k depende de los datos de entrada, porque si se cuenta con outliers o datos atípicos k debería tener un valor más alto.

A continuación, se explicará la implementación del algoritmo KNN en el código:}

Figura 6

Función para calcular la distancia

```python
def calcular_distancias(embedding_consulta, embeddings_train, fn_distancia):
    """
    Calcula la distancia entre la consulta y cada ejemplo del conjunto
    """
    distancias = []
    for i, emb in enumerate(embeddings_train):
        d = fn_distancia(embedding_consulta, emb)
        distancias.append((i, d))
    return distancias
```

Esta función compara el texto de entrada o también llamado consulta con cada uno de los textos que están almacenados como embeddings_train. y utiliza la función fn_distancia para calcular que tan lejos esta el texto de consulta respecto a los textos de entreamiento. Finalmente, esta función devuelve una lista donde se almacena i y d, donde i es índice y d la distancia de ese índice.

Figura 7

Función para seleccionar los k-vecinos

```python
def seleccionar_k_vecinos(distancias, k):
    """
    Ordena por distancia (ascendente) y retorna los k más cercanos
    """
    ordenadas = sorted(distancias, key=lambda x: x[1])
    return ordenadas[:k]
```

Esta función toma como argumentos la lista de distancias que devuelve la función anterior y el valor de k, que debe ser un número impar y se elige según los datos de entrada. La lista de las distancias se ordena de menor a mayor distancia, donde 0 es lo más cercano y en caso de que sea 0 quiere decir que es idéntico. Con [:k] se corta la lista, es decir se

queda solo con las distancias del valor de k, por ejemplo, si se eligió k=5, esta función devolverá los 5 más cercanos.

Figura 8

Función para votar la clase

```
def votar_clase(vecinos, clases_train):
    """
    Decisión por mayoría de votos entre los k vecinos
    """
    conteo = {}
    for idx, _ in vecinos:
        clase = clases_train[idx]
        conteo[clase] = conteo.get(clase, 0) + 1
    # Clase con mas votos
    return max(conteo, key=lambda c: conteo[c]), conteo
```

Esta función es la que permite tomar la decisión final, observa los vecinos seleccionados y mira que clase tienen y los va contando, cabe recalcar que este dataset cuenta con un total de 6 clases. Con el método *conteo.get(clase, 0) +1* si ve una clase le suma un punto, con max(conteo, …) se elige la clase que tuvo mayor cantidad de puntos o votos. Por ejemplo, si para la frase de consulta "*El artículo que recibí está roto*" tiene dos votos en la clase *producto_danano* y solo un voto en la clase *reclamo_entrega*, quiere decir que la predicción será *producto_danano.*

Figura 9

Función de clasificación

```python
def clasificar(embedding_consulta, embeddings_train, clases_train, k, fn_distancia):
    """
    Flujo completo de KNN para un unico punto de consulta
    """
    distancias = calcular_distancias(embedding_consulta, embeddings_train, fn_distancia)
    vecinos = seleccionar_k_vecinos(distancias, k)
    clase_pred, conteo = votar_clase(vecinos, clases_train)
    return clase_pred, vecinos, conteo
```

Esta función no realiza algún otro cálculo, ya que las tres funciones explicadas anteriormente realizan los cálculos necesarios, el objetivo de esta función es unir las tres funciones anteriores, ya que con distancias se midió a cada uno de los vecinos, con vecinos, se filtraron a los k mejores y con *clase_pred* se obtiene por votación la clase que ganó. Esta función lo que retorna es la clase ganadora, los vecinos y cuantos votos obtuvo.

Evaluación Hold-Out

Para evaluar el rendimiento del modelo predictivo se utiliza el método Hold-Out el cual toma el conjunto de datos y lo divide en dos conjuntos, el subconjunto más grande es el que se utiliza para el entrenamiento y el restante de los datos es el de test, que este permite evaluar la eficiencia del modelo. Comúnmente el dataset se divide en proporciones de la siguiente manera, 60% para train y 40% para test, 70% para train y 30% para test, en este caso se utilizó el 80% para entrenamiento y el 20% para evaluar el modelo, ya que es recomendable tener un conjunto de entrenamiento más grande porque así el modelo puede aprender mejor los patrones. La implementación en código de este método se aprecia en la figura 10, que posteriormente se explicará su funcionamiento, cabe recalcar que antes de repartir el set de datos es necesario aplicar una mezcla aleatoria para así reducir el sesgo en la muestra.

Figura 10

Función para evaluar la eficiencia del modelo

```
def evaluar_modelo(embeddings, clases, k, fn_distancia, proporcion_prueba=0.2, semilla=42):
    """
    Divide aleatoriamente en entrenamiento y prueba, ejecuta KNN y retorna la exactitud (accuracy)
    """
    random.seed(semilla)
    indices = list(range(len(embeddings)))
    random.shuffle(indices)

    corte = int(len(indices) * proporcion_prueba)
    indices_prueba = indices[:corte]
    indices_train  = indices[corte:]

    emb_train  = [embeddings[i] for i in indices_train]
    cls_train  = [clases[i]     for i in indices_train]
    emb_prueba = [embeddings[i] for i in indices_prueba]
    cls_prueba = [clases[i]     for i in indices_prueba]

    aciertos = 0
    for emb, cls_real in zip(emb_prueba, cls_prueba):
        pred, _, _ = clasificar(emb, emb_train, cls_train, k, fn_distancia)
        if pred == cls_real:
            aciertos += 1

    exactitud = aciertos / len(cls_prueba) if cls_prueba else 0.0
    return exactitud
```

Esta función permite verificar el funcionamiento del algoritmo KNN, en primer lugar, se preparan los datos y se mezclan esto se realiza con random, se le otorga una semilla la cual permite asegurar la aletoriedad. Con *random.shuffle* se mezclan los datos para poder asegurar que el modelo se entrene con todas las categorías y no aprenda más de una en caso de que el set de datos este organizado por clases, entonces el algoritmo aprendería muy bien a clasificar una sola clase y fallaría en las demás, por esta razón y como se mencionó anteriormente es necesario mezclar los datos.

Como segundo paso se divide el dataset en datos de entrenamiento y datos de prueba, como se puede apreciar en los argumentos de la función proporcion_prueba tiene un valor de 0.2, es decir los datos de prueba serán tan solo el 20% del total de dataset, de

esta manera se sobrentiende que para el entrenamiento se usará el 80% restante. Con corte se calcula cuantos datos se irán al subconjunto de test, con *índices_train* que corresponde al 80% son el conjunto de datos que utiliza el modelo para comparar y hallar los vecinos, finalmente con *índices_prueba*, son los casos nuevos que el algoritmo de KNN intentará clasificar.

En estas líneas de código *emb_train = [embeddings[i] for i in indices_train] cls_train = [clases[i] for i in indices_train]* se crean las listas de vectores o embeddings con sus etiquetas, que en este caso son las 6 clases que tiene el dataset; esto se realiza con los índices que fueron mezclados en el paso anterior. De la misma forma se crean las listas para los datos de prueba.

La evaluación se realiza en el bucle for, donde se toma un vector del grupo de prueba, es decir un vector que el modelo no conoce, se hace uso de la función clasificar para que el algoritmo busque los k vecinos más cercanos y los compare con los datos de entrenamiento. La predicción que arroje el algoritmo se compara con la etiqueta real y en caso de que sea la misma, es decir el modelo predijo la clase de reclamación correcta, se le anota un punto, que se almacena en la variable *aciertos*. Finalmente, se calcula el porcentaje de éxito o Accuracy devolviendo un porcentaje de éxito, en caso de que sea 100% puede ser una alerta de que el modelo cuenta con un problema de overfitting entonces siendo así se busca obtener un resultado cercano al 100%.

En el siguiente apartado se mostrarán los gráficos de le exactitud de cada métrica y la exactitud según la K elegida, además se interpretarán los resultados.

Gráficas

Las siguientes 3 figuras, es decir de la figura 11 a la 13, mediante un gráfico de barras permiten apreciar la precisión que tienen las 4 métricas frente a un k fijo, se probó con 3 valores, 3 – 5 y 7.

Figura 11

Comparación de exactitud con k=3

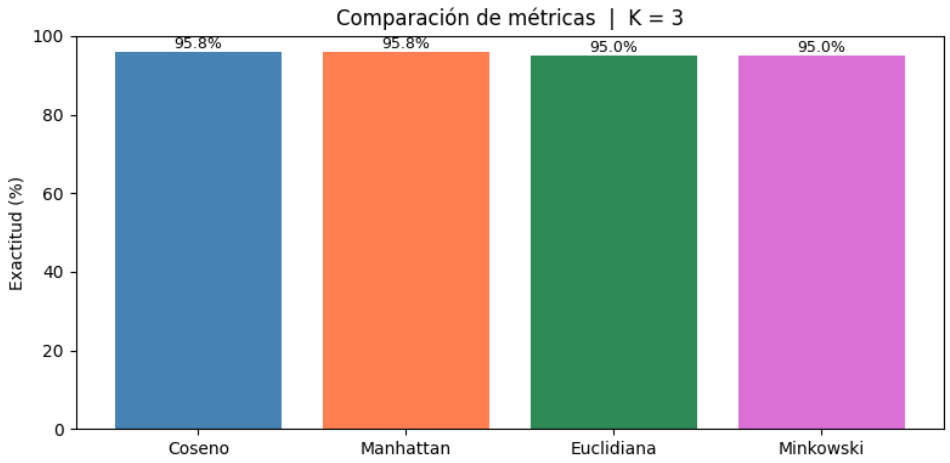


Figura 12

Comparación de exactitud con k=5

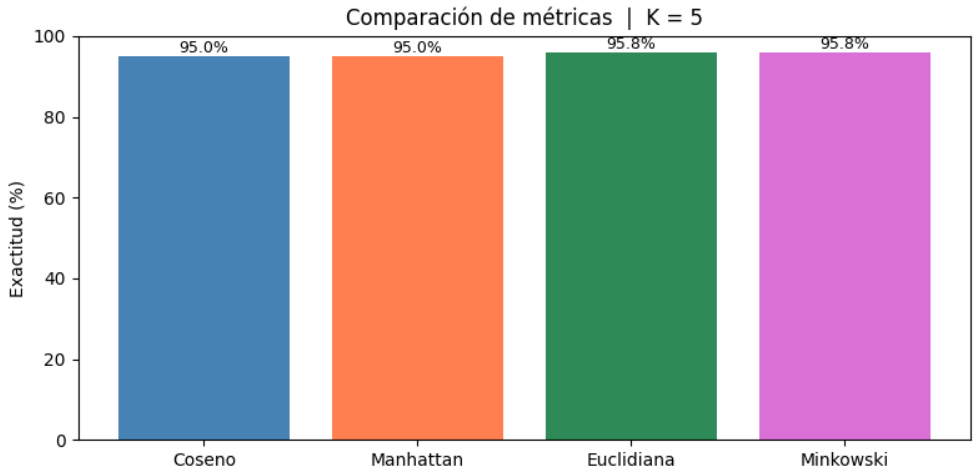
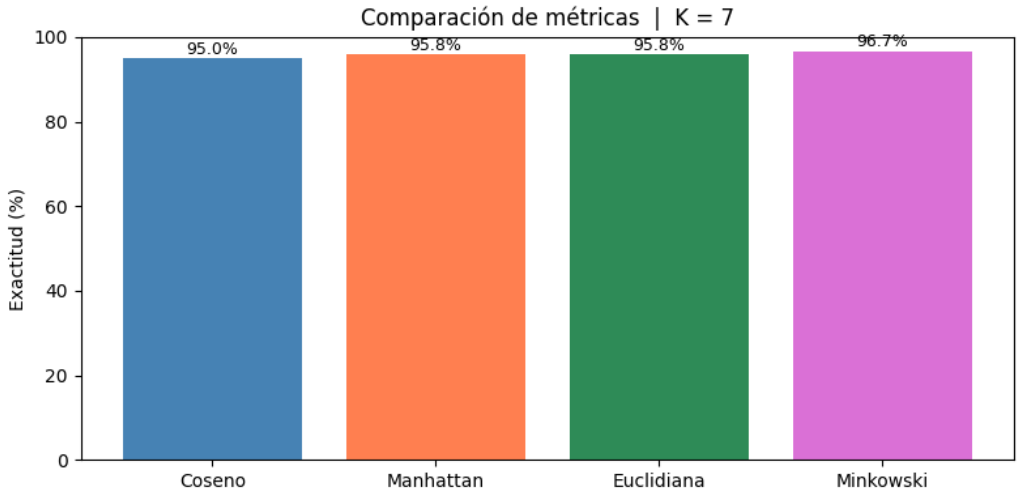


Figura 13

Comparación de exactitud con k=7



A partir de las figuras se puede apreciar que la exactitud por cada una de las métricas varía según el valor de k, en el caso en que k tiene el menor valor, es decir 3, la similitud de coseno y la distancia Manhattan tienen una mayor exactitud, con k=5 pasa lo contrario en este caso las métricas con mayor exactitud son la distancia Euclidiana y la distancia de Minkowski. Con el mayor valor de k es decir 7, se obtiene un mayor porcentaje de precisión en la métrica de Minkowski con un 96.7%, permitiéndonos identificar que deberíamos hacer uso de esta métrica para obtener la distancia entre los vectores y de igual forma la gráfica nos indica que la métrica con menor precisión es la similitud de coseno con un 95.0%, por lo que lo ideal sería usar alguna de las otras 3 métricas. De igual manera la gráfica permitió identificar que con un mayor valor de k, la exactitud aumenta.

Las siguientes figuras permiten observar vs el valor de k por cada una de las métricas.

Figura 14

Exactitud vs valores de K y métrica de coseno

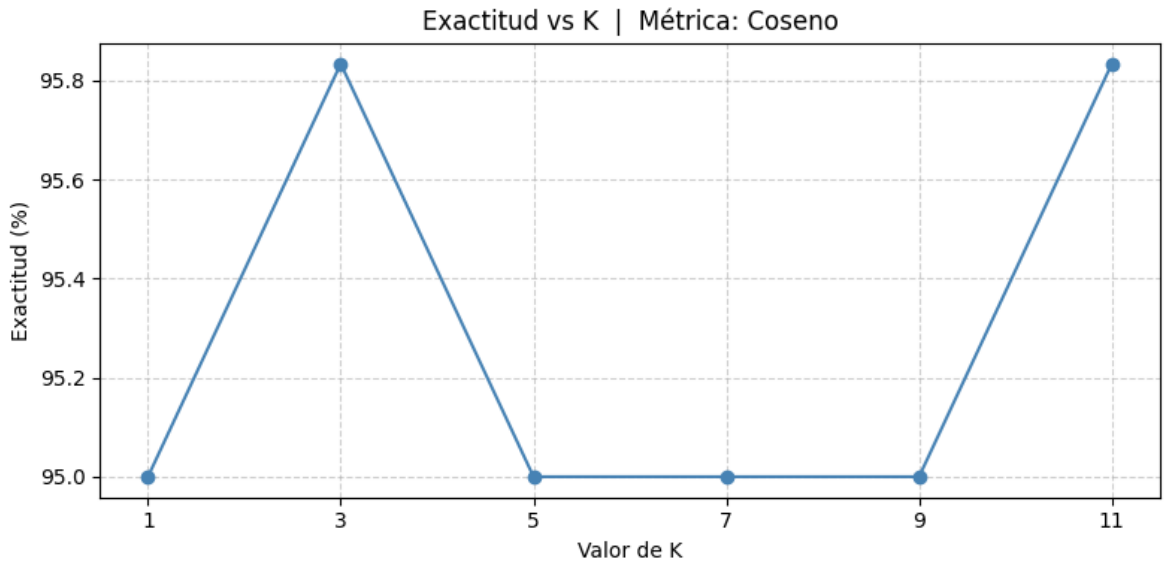


Figura 15

Exactitud vs valores de K y métrica de Manhattan

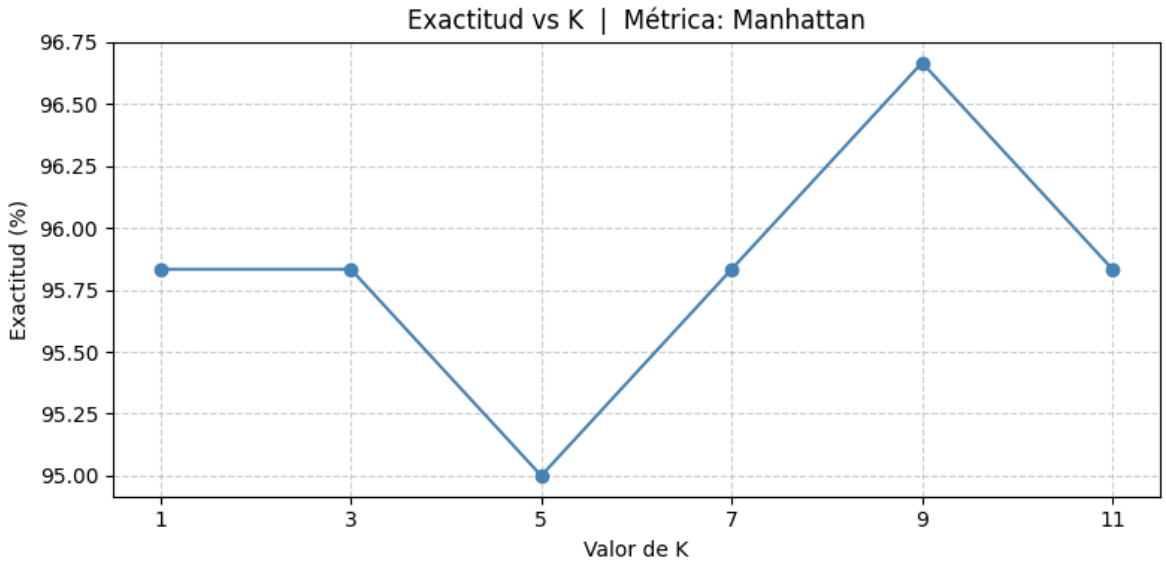


Figura 16

Exactitud vs valores de K y métrica Euclidiana

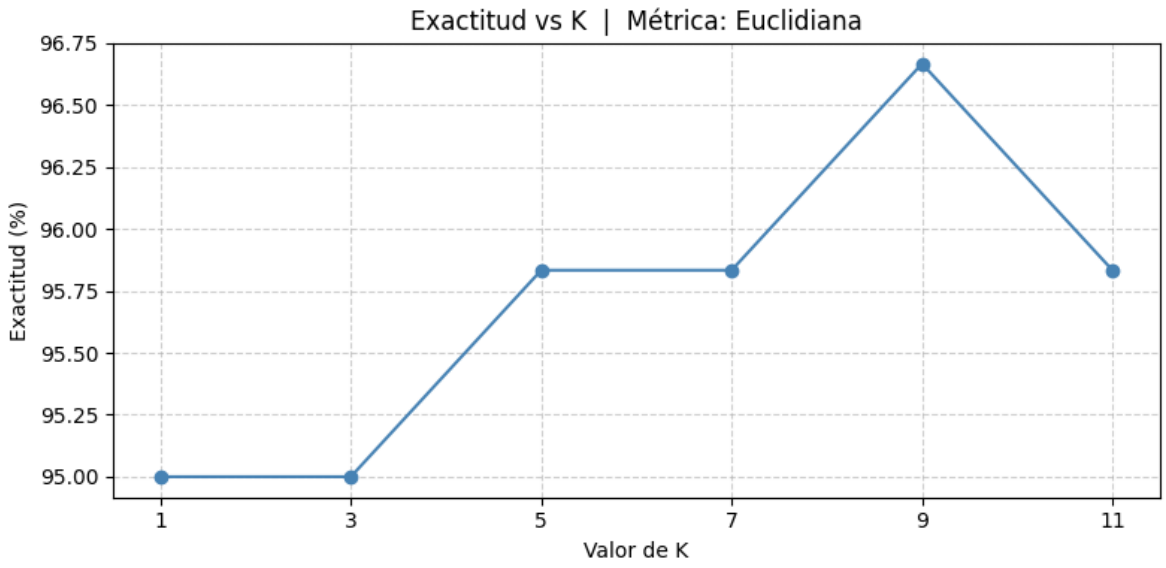
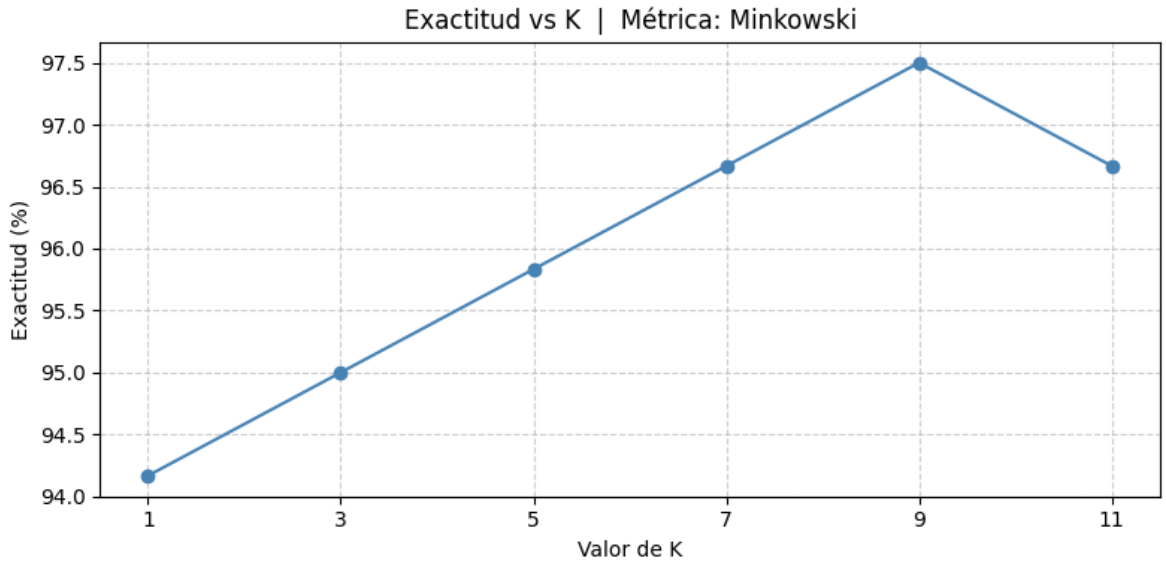


Figura 17

Exactitud vs valores de K y métrica de Minkowski



Evaluando cada una de las figuras anteriores y contrastando con las figuras 11 a la 13, se puede confirmar que la métrica de Minkowski tiene mayor efectividad a medida en que va aumentando el valor de k, si se observa la figura 17, se puede evidenciar que su crecimiento es prácticamente lineal hasta k=9, para estos gráficos se utilizaron 6 valores de k, estos valores son impares y son los siguientes: 1, 3, 5, 7, 9 y 11. La métrica que tiene mayor variabilidad en su exactitud es la similitud de coseno ya que para k=3 obtiene un buen porcentaje de exactitud, frente a otras métricas como la euclidiana o la de Minkowski, pero, su exactitud vuelve a bajar hasta k=11 que obtiene el mismo resultado que las métricas de Manhattan y Euclidiana.

Por esta razón y una vez más se confirma que la métrica que tiene mayor variabilidad y menor exactitud es la similitud de coseno, además, que las métricas de Manhattan y Euclidiana mejoran su precisión con valores de k más grandes, al igual que la Minkowski, pero esta a su vez es la métrica que otorga una exactitud mayor frente a las demás, para k=9, obtiene una exactitud de 97.5%, siendo así la métrica ganadora o elegida para el análisis.

Ejecución del algoritmo y pruebas

El código le solicita al usuario que ingrese el número correspondiente a la métrica con la que se desea calcular la distancia entre los vectores, también debe ingresar el valor de k, para evitar un empate este debe ser un número impar y el texto que el usuario desea clasificar, en la figura 18 se puede ver el primer ejemplo del uso del algoritmo creado.

Figura 18

Primer resultado

```
=== CLASIFICADOR KNN ===
Métricas disponibles:
  1. Coseno
  2. Manhattan
  3. Euclidiana
  4. Minkowski
Seleccione métrica (1-4): 4
Ingrese el valor de K: 7

Exactitud del modelo (hold-out 80/20): 96.67%

Ingrese el texto a clasificar: No puedo pagar con mi tarjeta debito

--- Resultado con métrica 'Minkowski' y K=7 ---
Clase predicha: problema_pago
Votos por clase: {'problema_pago': 7}

Vecinos más cercanos:
  [idx=391] Clase: problema_pago | Distancia: 1.5299 | Texto: el sistema rechazo mi tarjeta en bogota ayuda...
  [idx=250] Clase: problema_pago | Distancia: 1.5435 | Texto: tuve error al hacer el pago en bogota necesito solucion...
  [idx=456] Clase: problema_pago | Distancia: 1.5730 | Texto: e sistema rechazo mi tarjeta en bogota ya es mucho tiempo...
  [idx=305] Clase: problema_pago | Distancia: 1.5905 | Texto: el sistema rechazo mi tarjeta en funza nadie responde...
  [idx=158] Clase: problema_pago | Distancia: 1.5937 | Texto: tuve error al hacer el pago en chia nadie responde...
  [idx=291] Clase: problema_pago | Distancia: 1.5953 | Texto: el sistema rechazo mi tarjeta en chia ayuda...
  [idx=173] Clase: problema_pago | Distancia: 1.6051 | Texto: el sistema rechazo mi tarjeta en bogota necesito solucion...

¿Clasificar otro texto? (s/n): n
```

Para este ejemplo el algoritmo va a calcular la distancia entre vectores con la distancia de Minkowski, se le asigno a k el valor de 7, es decir el algoritmo va a buscar a los 7 vecinos más cercanos y el texto de entrada fue "*No puedo pagar con mi tarjeta debito*" que no está en el dataset y por su contexto pertenece a la clase *problema_pago*. El algoritmo obtuvo un 96.67% de exactitud lo que permite evidenciar que, si está clasificando adecuadamente los textos de entrada así no hagan parte del conjunto de entrenamiento, predijo efectivamente la clase *problema_pago* y los votos fueron por unanimidad. También mostró el índice de los 7 vecinos más cercanos junto con el texto y se puede apreciar que cada uno de los textos si tienen que ver con el texto de entrada, además la distancia obtenida para los 7 vecinos es no varía mucho, esta entre 1.5 y 1.6 lo que es un buen resultado, pero si es una distancia considerada, teniendo en cuenta resultados que se verán más adelante.

Para el siguiente ejemplo se usará k igual a 3 como se muestra en la figura 19, para encontrar los 3 vecinos más cercanos y se usará la entrada “Compré hace 8 días y mi pedido no ha llegado a Faca”.

Figura 19

Segundo resultado

```
PS D:\Users\Laura\Documents\Ingenieria de sistemas\IPA-2026\Machine Learning>  d:; cd 'd:\Users\Laura\Documents\Ingenieria de sistemas\IPA-2026\Machine Learning'
ata\Local\Programs\Python\Python313\python.exe' 'c:\Users\Laura\.vscode\extensions\ms-python.debugpy-2025.18.0\bundled\libs\debugpy\launcher' '59229' '--' 'd:\Us
nieria de sistemas\IPA-2026\Machine Learning\taller_5_ml_knn.py'
  2. Manhattan
  3. Euclidiana
  4. Minkowski
Seleccione métrica (1-4): 1
Ingrese el valor de K: 3

Exactitud del modelo (hold-out 80/20): 95.83%

Ingrese el texto a clasificar: Compre hace 8 dias y mi pedido no ha llegado a Faca

--- Resultado con métrica 'Coseno' y K=3 ---
Clase predicha: reclamo_entrega
Votos por clase: {'reclamo_entrega': 3}

Vecinos más cercanos:
  [idx=230] Clase: reclamo_entrega | Distancia: 0.2146 | Texto: mi pedido no ha llegado a cha desde mucho tiempo...
  [idx=4] Clase: reclamo_entrega | Distancia: 0.2176 | Texto: mi pedido no ha llegado a chia desde una semana ya es mucho ...
  [idx=59] Clase: reclamo_entrega | Distancia: 0.2219 | Texto: mi pedido no ha llegado a mosqra desde varios dias ya es muc...

¿Clasificar otro texto? (s/n): s

=== CLASIFICADOR KNN ===
Métricas disponibles:
  1. Coseno
  2. Manhattan
  3. Euclidiana
  4. Minkowski
Seleccione métrica (1-4): 4
Ingrese el valor de K: 3

Exactitud del modelo (hold-out 80/20): 95.00%

Ingrese el texto a clasificar: Compre hace 8 dias y mi pedido no ha llegado a Faca

--- Resultado con métrica 'Minkowski' y K=3 ---
Clase predicha: reclamo_entrega
Votos por clase: {'reclamo_entrega': 3}

Vecinos más cercanos:
  [idx=509] Clase: reclamo_entrega | Distancia: 0.9062 | Texto: mi pedido no ha legado a chia desde una semana q pasa con es...
  [idx=491] Clase: reclamo_entrega | Distancia: 0.9103 | Texto: mi pedido no ha llegado a chia desde dias q pasa con esto...
  [idx=230] Clase: reclamo_entrega | Distancia: 0.9170 | Texto: mi pedido no ha llegado a cha desde mucho tiempo...
```

Con el mismo texto se hizo la prueba con la métrica (1) similitud por coseno, la cuál tiene una precisión del 95.8% para k=3 como se puede ver en la figura 14, es el k donde tiene mayor precisión por eso se eligió este para probar que tanto varía frente a otras métricas como la de (4) Minkowski que a medida que crece k, va aumentando su precisión. Si se observa la figura 19, los 3 vecinos más cercanos usando la métrica de coseno, la distancia esta entre 0.21 y 0.22, es un excelente resultado si lo comparamos con Minkowski

que la distancia varía entre 0.90 y 0.91, es una gran diferencia a pesar de que ambas métricas coinciden en la clase a la que pertenece la reclamación que es *reclamo_entrega*, los textos que entregan cada métrica si tienen una gran relación con el texto de prueba y de hecho ambas métricas coinciden con el texto del id 230 que dice "*Mi pedido no ha llegado a cha desde mucho tiempo*", que realmente utiliza casi las mismas palabras del texto de prueba pero con otra ciudad de destino. En la siguiente figura se realizará la prueba con un texto que no tiene relación a una compra, para probar que el algoritmo esta clasificando adecuadamente.

Figura 20

Prueba tres con frase diferente al contexto

```
¿Clasificar otro texto? (s/n): s

=== CLASIFICADOR KNN ===
Métricas disponibles:
  1. Coseno
  2. Manhattan
  3. Euclidiana
  4. Minkowski
Seleccione métrica (1-4): 2
Ingrese el valor de K: 7

Exactitud del modelo (hold-out 80/20): 95.83%

Ingrese el texto a clasificar: La casa es verde y se encuentra lejos de aqui

--- Resultado con métrica 'Manhattan' y K=7 ---
Clase predicha: reclamo_entrega
Votos por clase: {'reclamo_entrega': 5, 'producto_danado': 2}

Vecinos más cercanos:
  [idx=297] Clase: reclamo_entrega | Distancia: 75.5538 | Texto: no recibo el paquete en facatativa y ya paso dias ya reclame...
  [idx=406] Clase: reclamo_entrega | Distancia: 75.8745 | Texto: no recibo el paqte en mosqra y ya paso mas de una semana q p...
  [idx=549] Clase: reclamo_entrega | Distancia: 76.3388 | Texto: no recibo el paquete en funza y ya paso mas de una semana po...
  [idx=20] Clase: reclamo_entrega | Distancia: 76.5527 | Texto: no recibo el paqte en chia y ya paso mas de una semana neces...
  [idx=83] Clase: producto_danado | Distancia: 76.5796 | Texto: lo qu compre llego en mal estado a funza ya reclame antes...
  [idx=204] Clase: producto_danado | Distancia: 76.6082 | Texto: lo q compre llego en mal estado a bogota ayuda...
  [idx=385] Clase: reclamo_entrega | Distancia: 76.6135 | Texto: no recibo el paqte en chia  ya paso mas de una semana ayuda...
```

Se tomo el texto "La casa es verde y se encuentra lejos de aquí" que no tiene que ver con el contexto para el cual fue entrenado el algoritmo que es sobre un ecommerce, este describe una casa solamente, por ende el algoritmo tendría que mostrar una distancia grande y confundirse, tal y como ocurrió en la ejecución donde la distancia para los 7

vecinos fue entre 75.5 y 76.6, confirmando así que el algoritmo funciona adecuadamente para textos que tienen que ver con el entrenamiento del algoritmo, no está dando resultados porque aprendió mucho sobre los datos de entrenamiento, realmente el busco la coincidencia que podía llegar a tener el texto de prueba con los datos de entrenamiento y mostró los 7 vecinos más cercanos, por obvias razones se confundió y estos no tienen relación con la frase de prueba pero el algoritmo evaluó que esos 7 textos eran los que más se acercaban. De igual forma la predicción de la clase no fue por unanimidad como en los casos anteriores, aquí el algoritmo puso dos clases tentativas que fue *reclamo_entrega* y *producto_danado*, con 5 y 2 votos respectivamente, por lo que el algoritmo se decantó hacia la primera clase que tuvo la mayor cantidad de votos.

Figura 21

Prueba métrica Euclidiana

```
=== CLASIFICADOR KNN ===
Métricas disponibles:
  1. Coseno
  2. Manhattan
  3. Euclidiana
  4. Minkowski
Seleccione métrica (1-4): 3
Ingrese el valor de K: 9

Exactitud del modelo (hold-out 80/20): 96.67%

Ingrese el texto a clasificar: No me gusto el producto que recibi quiero devolverlo

--- Resultado con métrica 'Euclidiana' y K=9 ---
Clase predicha: devolucion_reembolso
Votos por clase: {'devolucion_reembolso': 6, 'producto_danado': 2, 'reclamo_entrega': 1}

Vecinos más cercanos:
  [idx=594] Clase: devolucion_reembolso | Distancia: 2.5798 | Texto: quiero devolver el producto comprado en bogota...
  [idx=129] Clase: devolucion_reembolso | Distancia: 2.6162 | Texto: quiero devolver el producto comprado en bogota ya reclame an...
  [idx=66] Clase: devolucion_reembolso | Distancia: 2.6859 | Texto: quiero devolver el producto comprado en funza ya reclame ant...
  [idx=576] Clase: devolucion_reembolso | Distancia: 2.7111 | Texto: quiero devolver el producto comprado en bogota ya es mucho t...
  [idx=220] Clase: producto_danado | Distancia: 2.7448 | Texto: el paquete llego pero el producto esta malo en bogota...
  [idx=206] Clase: reclamo_entrega | Distancia: 2.7451 | Texto: no entregaron el producto en chia ya reclme antes...
  [idx=64] Clase: devolucion_reembolso | Distancia: 2.7611 | Texto: quiero devolver el producto comprado en chia porfa urente...
  [idx=192] Clase: devolucion_reembolso | Distancia: 2.7667 | Texto: quiero devolver el producto comprado en funza necesito soluc...
  [idx=234] Clase: producto_danado | Distancia: 2.7700 | Texto: el paquete llego pero el producto esta malo en facatativa...
```

Para probar la métrica de distancia Euclidiana que obtiene muy buenos resultados cuando k=9 con una precisión del 96.67% se tomo el siguiente texto “No me gusto el

producto quiero devolverlo”, se obtuvo buenos resultados la distancia no es tan grande esta entre 2.5 y 2.7, la votación no fue por unanimidad, porque el algoritmo tomo 3 clases para la votación, *devolución_reembolso, producto_danado* y *reclamo_entrega*, tiene sentido que tomara estas 3 clases en cuenta porque la frase al decir no me gusto el producto, es algo ambiguo porque se puede interpretar como que no le gustó el producto porque esta dañado, pero de igual forma el algoritmo clasificó eficazmente el texto de prueba, por mayoría de votos ganó la clase *devolución_reembolso*, siendo esta la clase correcta.

Finalmente, para comprobar la métrica con mayor exactitud, se toma la frase anterior para a su vez comparar que tanto varían los resultados entre métricas.

Figura 22

Prueba métrica Minkowski

```
=== CLASIFICADOR KNN ===
Métricas disponibles:
  1. Coseno
  2. Manhattan
  3. Euclidiana
  4. Minkowski
Seleccione métrica (1-4): 4
Ingrese el valor de K: 9

Exactitud del modelo (hold-out 80/20): 97.50%

Ingrese el texto a clasificar: No me gusto el producto que recibi quiero devolverlo

--- Resultado con métrica 'Minkowski' y K=9 ---
Clase predicha: devolucion_reembolso
Votos por clase: {'devolucion_reembolso': 7, 'reclamo_entrega': 1, 'producto_danado': 1}

Vecinos más cercanos:
  [idx=594] Clase: devolucion_reembolso | Distancia: 1.1198 | Texto: quiero devolver el producto comprado en bogota...
  [idx=129] Clase: devolucion_reembolso | Distancia: 1.1401 | Texto: quiero devolver el producto comprado en bogota ya reclame an...
  [idx=66] Clase: devolucion_reembolso | Distancia: 1.1688 | Texto: quiero devolver el producto comprado en funza ya reclame ant...
  [idx=576] Clase: devolucion_reembolso | Distancia: 1.1820 | Texto: quiero devolver el producto comprado en bogota ya es mucho t...
  [idx=206] Clase: reclamo_entrega | Distancia: 1.1926 | Texto: no entregaron el producto en chia ya reclme antes...
  [idx=156] Clase: devolucion_reembolso | Distancia: 1.1990 | Texto: quiero devolver el producto comprado en chia ayuda...
  [idx=64] Clase: devolucion_reembolso | Distancia: 1.2005 | Texto: quiero devolver el producto comprado en chia porfa urente...
  [idx=220] Clase: producto_danado | Distancia: 1.2067 | Texto: el paquete llego pero el producto esta malo en bogota...
  [idx=192] Clase: devolucion_reembolso | Distancia: 1.2090 | Texto: quiero devolver el producto comprado en funza necesito soluc...
```

Esta métrica tiene la mayor exactitud como se pudo apreciar en las gráficas anteriores, su precisión es del 97.50% por lo que se decidió comparar los resultados con la similitud Euclidiana, ambas métricas votaron por las mismas 3 clases, pero esta vez la clase

ganadora tuvo un voto más (6) y *producto_danado* tan solo un voto, permitiendo confirmar que el algoritmo clasifica adecuadamente, que puede confundirse un poco y votar a una clase que no es, como lo es *producto_danado* pero esta bien, porque la frase también se puede interpretar en parte de esa manera. La distancia fue menor en esta métrica estuvo entre 1.1 y 1.2, devolvió casi los mismos vecinos, solo varió el de id 234 perteneciente a la clase *producto_danado* que en esta métrica solo había 1 vecino de esa clase y fue el id 220, esto permite ver una consistencia entre las métricas.

Conclusiones

Teniendo en cuenta cada una de las pruebas realizadas y los gráficos que se obtuvieron, se puede evidenciar que el algoritmo funciona con bastante exactitud los resultados que arroja son coherentes con las frases de prueba, puede que llegue a confundirse con ciertas frases, pero tiene su justificación, la frase puede tener varias interpretaciones y al final el algoritmo votó por la clase que era y gano por mayoría de votos. También se evidenció consistencia entre las métricas. al poner la misma frase se obtuvieron casi los mismos vecinos, la distancia varió, pero no de una forma significante.

De igual manera, se pudo observar que métrica obtenía mayor exactitud respecto a 6 valores de k, para valores muy pequeños se recomendaría el uso de la similitud de coseno, que tiene un porcentaje mayor de exactitud respecto a las otras métricas y para valores de k grandes como 9, la métrica adecuada es la distancia de Minkowski que de todas las k y métricas es el valor más alto en exactitud. Por esta razón se escogieron los valores de k para las diferentes pruebas que se realizaron.

Bibliografía

Krantz, T., & Jonker, A. (2025, 28 noviembre). Similitud de coseno. *IBM*. https://www.ibm.com/es-es/think/topics/cosine-similarity

Barnard, J. (2025, 27 noviembre). Embedding. *¿Qué es embedding?* https://www.ibm.com/mx-es/think/topics/embedding

Railey Brandon. (2019). Python Machine Learning. Brandon Railey. https://bibliotecaucundinamarca-odilotk-es.ucundinamarca.basesdedatosezproxy.com/info/python-machine-learning-a-practical-beginners-guide-to-understanding-machine-learning-deep-learning-and-neural-networks-with-python-scikit-learn-tensorflow-and-keras-02282103